Epic / Checking ClickUp job tasks

Okay can you seen in Clickup what my jobs are ?

Loaded tools, used ClickUp integration

Here are your current ClickUp tasks, Matthew:

In Progress

- Draw final system architecture diagram showing all services / ports / protocols / directions

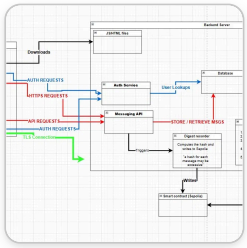
To Do

- Define C++ component scope: CLI client with TLS sockets / OOP message store / CMake build
- Document all external connections in architecture doc: Sepolia RPC / DB / Redis / Blockchain

Done ✅

- Set up Docker Compose skeleton with FastAPI / PostgreSQL / Redis services

All four tasks are in the **Sprint 1: Planning & Architecture** list. The Docker Compose setup is already done — nice work! The architecture diagram is your active task, and the two "to do" items are still to be started.



Yeah I need to improve this - what needs to be added / changed?

Looking at your diagram against the project requirements and your codebase, here's what needs to be added or fixed:

Missing entirely

- **Redis** — it's a core service in your stack (blockchain write queue via Redis Streams), but it's not shown at all. The flow should be: Messaging API → Redis Stream → Digest Recorder → Sepolia
- **PostgreSQL** is labelled just "Database" — fine, but worth labelling it properly since the spec asks for external connections to be documented
- **Ports and protocols** — your task specifically says "all services / ports / protocols / directions". You need to label ports: 8000 (FastAPI), 5432 (Postgres), 6379 (Redis), 443 (HTTPS/TLS)
- **X3DH / Double Ratchet key exchange flow** — the crypto design is a big part of the mark. Even a simplified note showing that E2EE happens client-side before anything hits the server would help
- **The `/api/auth/user/{u}/keybundle` endpoint** — clients need to fetch prekeys before sending. This flow isn't shown

Things to fix

- The **TLS connection arrow** from the C++ client goes nowhere useful — it should connect to the backend server boundary, not float off the bottom of the diagram
- **JS/HTML files being "downloaded"** from the backend is a bit misleading — in your setup it's a static file server or just opened locally, worth clarifying
- The **Blockchain Verification Page** is shown as a separate floating component with no network connection shown to the smart contract or the internet — it needs an arrow showing it's a standalone web page querying Sepolia RPC directly
- Auth Service and Messaging API look like two separate servers — clarify they're both routes within the **single FastAPI app**

Nice to have for full marks

- A legend explaining your colour coding (red = auth, blue = user lookups, green = TLS, etc.)
- Label the **Sepolia RPC endpoint** (Infura/Alchemy) as the intermediary between your backend and the smart contract — you don't call the contract directly, you go through an RPC provider
- Show that **private keys never leave the client** — a small note on both client boxes would directly address the E2EE requirement

The biggest gaps for your specific task are the missing Redis service and the missing port/protocol labels. Want me to redraw this as a cleaner version?

excerpt_from_previous_claude_message.txt
1 line
TXT

wdym label them? should i just enter them into the textbox of the containers?

Yeah exactly — just add the port number into the label of each container/box. So instead of just "Database" it would be something like:

"PostgreSQL :5432"

And the FastAPI backend box would be **"Backend Server :8000"** etc.

For the connection arrows, you can also just add a small text label on the arrow itself — so the HTTPS arrows would say **"HTTPS :443"** on the line. Most diagramming tools let you double-click an arrow to add a label.

So two places to add them:

1. **On the boxes** — the port the service listens on
2. **On the arrows** — the protocol and port being used for that connection

That way anyone reading the diagram immediately knows what's running where and how things talk to each other, which is exactly what the network architecture documentation requirement is asking for.

but just to clarify these portd are internal... right? like do i rlly need to clairfy all this

Good question — yes, most of them are internal (container-to-container within Docker Compose), but it's still worth labelling because:

The spec literally says it — "connections to external services are documented" and your task says "all services / ports / protocols / directions". The marker will be looking for it.

What's actually external vs internal:

- `:5432` (Postgres) — internal only, never exposed publicly ✅
- `:6379` (Redis) — internal only ✅
- `:8000` (FastAPI) — internal, but exposed to clients via reverse proxy or directly
- `:443` HTTPS — **this is external**, client to server
- Sepolia RPC (Infura/Alchemy) — **external**, your backend calling out to the internet
- Smart contract reads from the verification page — **external**

So the honest answer is you don't need to obsess over the internal ones, but a couple of lines in your architecture doc saying "Postgres and Redis are only reachable within the Docker network, not exposed publicly" would actually score you points under **Security Misconfiguration** in Burkley's rubric — it shows you thought about it.

The external connections (HTTPS :443, Sepolia RPC endpoint) definitely need to be labelled and documented properly.

> should the auth service box be inside the messaging api box? or should they both be in a collective FastAPI : 8000 box

They should both be inside a collective **"FastAPI :8000"** box — because that's what's actually true. They're just routers within the same FastAPI app (`main.py`), not separate services:

```python
# from your main.py
app.include_router(auth.router, prefix="/api/auth")
app.include_router(messages.router, prefix="/api/messages")
app.include_router(blockchain.router, prefix="/api/blockchain")
```

So your diagram should look something like:

```
┌─ FastAPI :8000 ──────────────────┐
│  ┌─ Auth Service ──────────────┐  │
│  │  /api/auth/*                │  │
│  └─────────────────────────────┘  │
│  ┌─ Messaging API ─────────────┐  │
│  │  /api/messages/*            │  │
│  └─────────────────────────────┘  │
│  ┌─ Blockchain Router ─────────┐  │
```

```
|  |  /api/blockchain/*          |  |
|  └──────────────────────────────┘  |
└────────────────────────────────────┘
```

You could even add a **Blockchain router** box in there too since you have one — that would show the marker you understand the full API surface and it maps directly to your actual code structure.

wdym by blockchain router\? i thought everythign that oes through messaging api gets sent to digest recorder thing

Good catch — let me clarify. You're right that the message flow goes through the Messaging API → Digest Recorder → Sepolia. The Blockchain Router (`/api/blockchain/*`) is actually a **separate read-only endpoint** for checking/verifying on-chain status — things like:

- `GET /api/blockchain/status/{id}` — check if a message hash has been confirmed on-chain
- `GET /api/blockchain/verify/{id}` — tamper-evidence check

So the flows are actually different:

Write path (what you already have):

```
Messaging API → Digest Recorder → Redis → Sepolia
```

Read path (what's missing from your diagram):

```
Client → Blockchain Router → queries Sepolia RPC → returns status
```

And the **Verification Page** is separate again — it's a standalone webpage that talks to Sepolia directly without going through your backend at all.

So yes, keep your diagram as-is for the write path, but it might be worth adding the blockchain router box inside FastAPI just to show the read/verify flow exists too. It maps directly to `blockchain.py` in your routers folder.

But honestly if you're tight on time, the write path is the more important one to get right — the read endpoints are currently just stubs in your code anyway.

excerpt_from_previous_claude_message.txt
1 line
TXT

hmm yeah this bit - do we know what we setlled on? i think it was the signal method - maybe do you want to look at the lecture slides where he explained it and gather what the flow is supposed to look like? i mean you should already know what best practice is.

You've definitely settled on the Signal Protocol (X3DH + Double Ratchet) — it's all over your codebase, `crypto_setup.txt`, and the README. I don't need the lecture slides for this, it's well-documented.

For your architecture diagram, you don't need to show the full protocol detail (that belongs in the crypto design document). Just a simple note on the diagram showing E2EE happens client-side. Something like:

On the client boxes, add a small note:

> "Keys generated & stored locally. Plaintext never leaves device."

On the arrows between clients and Messaging API, label them:

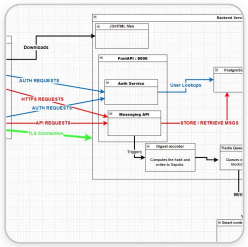
> "Ciphertext only (AES-256-GCM)"

A small E2EE flow note somewhere on the diagram:

```
Registration:  Client generates IK + SPK + OPKs → uploads public keys only
First message: Client fetches recipient keybundle → runs X3DH → seeds ratchet
Each message:  Double Ratchet derives fresh key → AES-256-GCM encrypt → send ciphertext
Server sees:   ciphertext + metadata only, never plaintext
```

That's honestly enough for the architecture diagram. The detailed X3DH/Double Ratchet walkthrough with diagrams is what goes in the **crypto design document** (Eoin's 2-6 page PDF requirement) — which is a separate deliverable anyway.

Would you like help drafting that crypto design document? That's where the real crypto marks are.



is this fine? i dont rlly want to spend two long on this

Yeah this is looking good, a few quick things:

Fix these (5 mins):

- The **TLS Connection** green arrow still goes nowhere — point it at the Backend Server box
- **Messaging API** should also have an arrow to PostgreSQL for storing/retrieving messages, not just Auth Service

Could skip if tight on time:

- The E2EE note on the client boxes
- Blockchain router box inside FastAPI
- Port labels on the arrows themselves

Honestly for the architecture diagram task this is **solid enough**. The Redis Queue :6379 addition is good, PostgreSQL :5432 is good, FastAPI :8000 box with services inside is correct.

Don't spend more than another 10-15 mins on it — the crypto design doc and actually implementing stuff will get you far more marks.

AI_Prompt_Artefacts.docx
57 lines
DOCX

Look back through this conversation and identify every message I (the human) sent that could serve as an AI prompt artefact for a university submission. I need prompts that demonstrate: - Security-driven thinking (e.g. specifying exact crypto parameters, security properties, failure modes) - Critical judgement (e.g. constraining scope, correcting a previous output, rejecting something) - Technical specificity (e.g. naming exact files, algorithms, RFCs, line numbers) For each one you find, return: 1. The exact text of my message Skip anything that's just a short question or command like "give me the cmds" or "yes do that". I only want substantive prompts. can you add them to this file , just the straight prompts

Check the SKILL.md for docx >

Now let me look through this current conversation for qualifying prompts from you, then add them to the doc.

Ran 4 commands, viewed a file >

Good. Now I can see the document structure clearly. From this conversation, the qualifying prompts are:

1. The architecture diagram feedback prompt (uploading the image and asking what needs to be added/changed)
2. The "should the auth service box be inside the messaging api box?" correction prompt
3. The Signal Protocol prompt asking to review the lecture slides
4. The meta prompt asking to add prompts to the file (already in there as Prompt 10)

Let me add those as Prompts 11, 12, and 13:

Edited a file, ran a command, read a file >

Added three new prompts (11, 12, 13) from this conversation:

- **Prompt 11** — uploading the architecture diagram and asking what needs to be improved (shows architectural thinking and security awareness)
- **Prompt 12** — correcting the diagram structure by questioning whether Auth Service and Messaging API should be separate or under a single FastAPI box (critical judgement / technical accuracy)
- **Prompt 13** — asking to verify the Signal Protocol implementation against the lecture and best practice (security-driven thinking, naming a specific protocol)

Ai prompt artefacts
Document · DOCX
Download